

Implementação e Análise Comparativa entre TCP e UDP Confiável para Transferência de Arquivos em Cenários com Perda e Latência Simuladas

Pedro Henrique Silva Rodrigues¹
(MATRÍCULA: 20249011446)

¹ Universidade Federal do Piauí (UFPI) – Campus Senador Helvídio
Nunes de Barros (CSHNB) Picos – PI – Brasil

pedro.rodrigues.pr@ufpi.edu.br

Resumo. Este trabalho apresenta uma implementação e análise comparativa entre os protocolos TCP e UDP Confiável (R-UDP) para transferência de arquivos. O R-UDP emprega um mecanismo Stop-and-Wait com numeração de sequência, reconhecimento (ACK), timeout, retransmissão e verificação de integridade via CRC32 na camada de aplicação. Ambos os modos foram submetidos a três cenários de rede simulados com a ferramenta TC (atraso e perda de pacotes) em ambiente Docker. As métricas de vazão e duração foram coletadas por benchmark automatizado e validadas cruzadamente com capturas TCPDump/Wireshark por meio de um cabeçalho de autenticação X-Custom-Auth. Os resultados mostram que o TCP supera significativamente o R-UDP em todos os cenários, especialmente sob perda e latência elevadas, evidenciando a eficiência do controle de fluxo por janela deslizante e do timeout adaptativo do TCP frente à abordagem simplificada Stop-and-Wait do R-UDP.

Palavras-chave: TCP; UDP Confiável; Transferência de Arquivos; Simulação de Rede.

1. Introdução

As redes de computadores constituem a espinha dorsal dos sistemas modernos, viabilizando a comunicação entre dispositivos e sustentando aplicações que exigem entrega confiável de dados, baixa latência e utilização eficiente dos recursos de infraestrutura. Nesse contexto, os protocolos da camada de transporte desempenham um papel determinante no equilíbrio entre desempenho, confiabilidade e escalabilidade, influenciando diretamente a experiência do usuário final e a viabilidade técnica de sistemas críticos. A compreensão do comportamento desses protocolos sob condições adversas de rede é, portanto, essencial para o projeto de aplicações robustas e eficientes.

As aplicações de rede operam frequentemente em ambientes sujeitos a latência variável, perda de pacotes e flutuações de carga, cenários que podem comprometer severamente a eficiência da comunicação. Nesse cenário, o TCP (*Transmission Control Protocol*) destaca-se por oferecer entrega confiável e ordenada por meio de mecanismos como controle de fluxo por janela deslizante, controle de congestionamento, reconhecimento positivo (ACK) e retransmissão de segmentos perdidos. Em contraste, o UDP (*User Datagram Protocol*) prioriza simplicidade e baixo *overhead*, porém não provê qualquer garantia de entrega, ordenação ou retransmissão automática.

Este trabalho propõe a implementação e a análise comparativa entre o TCP e um protocolo UDP Confiável (R-UDP) para transferência de arquivos, submetendo ambos a cenários de rede controlados simulados, variando atraso e taxa de perda de pacotes. A comparação fundamenta-se em métricas objetivas de vazão (*throughput*) e tempo de transferência, visando evidenciar as vantagens e limitações de cada abordagem em condições realistas de operação.

1.1. Objetivos

Este trabalho tem como objetivos:

1. **Implementar módulo de transferência sobre TCP:** desenvolver um par cliente-servidor capaz de transferir arquivos utilizando o protocolo TCP nativo, servindo como base de desempenho para as comparações.
2. **Implementar módulo de transferência sobre R-UDP:** construir uma camada de confiabilidade sobre UDP na aplicação, aplicando mecanismo *Stop-and-Wait* com numeração de sequência, ACK, *timeout*, retransmissão e verificação de integridade via CRC32.
3. **Simular cenários adversos de rede:** reproduzir condições controladas de latência e perda de pacotes utilizando as ferramentas tc / netem em ambiente Docker, abrangendo três cenários: baixa latência sem perdas, latência média com perda moderada e alta latência com perda elevada.
4. **Comparar o desempenho entre TCP e R-UDP:** analisar e contrastar os resultados obtidos em todos os cenários, identificando vantagens, limitações e comportamentos distintos de cada abordagem sob diferentes condições de rede.

1.2. Escopo e Delimitações

O escopo deste trabalho está delimitado aos seguintes aspectos:

- **Linguagem e ferramentas:** o trabalho foi implementado em Python 3.10+, utilizando Docker Compose v2 para orquestração dos *containers*, tc/netem para simulação de condições de rede, TCPDump e Wireshark/tshark para captura e validação dos pacotes, e Pandas/Matplotlib para análise estatística e geração dos gráficos.
- **Protocolos implementados:** foram implementados dois modos de transferência de arquivos: (i) sobre TCP nativo, e (ii) sobre UDP com camada de confiabilidade na aplicação, denominado R-UDP.
- **Mecanismos de confiabilidade do R-UDP:** a camada de confiabilidade sobre UDP emprega o algoritmo *Stop-and-Wait*, com numeração de sequência, ACK, temporizador fixo de 2 segundos, retransmissão de pacotes (até 1.000 tentativas), e verificação de integridade por CRC32.
- **Ambiente experimental:** os experimentos são executados em ambiente Docker com dois *containers* interconectados por uma rede, um servidor (receptor) e um cliente (transmissor), ambos na mesma máquina hospedeira. Um cabeçalho de autenticação X-Custom-Auth (SHA-256) é incluído em todas as mensagens para permitir a identificação dos fluxos nas capturas de rede.
- **Cenários de perda e latência:** três cenários de rede são simulados no *container* cliente por meio do comando tc qdisc netem: Cenário A (atraso de 10 ms, 0% de perda), Cenário B (50 ms, 5% de perda) e Cenário C (100 ms, 10% de perda).

- **Métricas analisadas:** as métricas coletadas em cada transferência são: vazão (em Mbps) e duração da transferência (em segundos). As métricas são registradas em arquivos CSV e posteriormente sumarizadas por estatísticas descritivas (mínimo, médio, máximo e desvio padrão) para cada combinação de modo e cenário.

1.3. Questões a Serem Respondidas

Este trabalho busca responder às seguintes questões fundamentais:

1. Como a curva de *throughput* do TCP se comportou em comparação ao seu R-UDP sob 10% de perda?
2. Ao analisar o Wireshark/TCPDump, qual foi o *overhead* (em *bytes*) introduzido pelos cabeçalhos do seu protocolo R-UDP?
3. Houve discrepância entre o tempo medido pelo Python e o registrado no Wireshark/TCPDump? Justifique.

2. Fundamentação Teórica

Esta seção apresenta os conceitos fundamentais necessários para a compreensão do trabalho, abordando os dois principais protocolos da camada de transporte, TCP e UDP, bem como a implementação de uma camada de confiabilidade sobre UDP denominada R-UDP. São discutidos os mecanismos de controle de fluxo, entrega confiável, ordenação e verificação de integridade que distinguem cada abordagem, fornecendo a base teórica para a análise comparativa realizada nos experimentos.

2.1. Protocolo TCP (Transmission Control Protocol)

O Protocolo de Controle de Transmissão (TCP) é um dos principais protocolos da camada de transporte da pilha TCP/IP, oferecendo um serviço orientado à conexão, confiável e com entrega ordenada de dados. Antes de qualquer transmissão, o TCP estabelece uma conexão lógica entre as partes por meio do mecanismo de *three-way handshake*, no qual são negociados números de sequência iniciais e parâmetros de janela.

Na transmissão, cada segmento enviado é numerado sequencialmente e confirmado por meio de ACKs cumulativos. Se um segmento não for confirmado dentro de um intervalo de tempo chamado RTO (*Retransmission Timeout*), calculado de forma adaptativa com base no RTT (*Round-Trip Time*) estimado, o TCP o retransmite automaticamente.

O controle de fluxo é realizado por meio de uma janela deslizando, na qual o receptor anuncia o tamanho de sua janela de recepção, limitando a quantidade de dados que o transmissor pode enviar sem confirmação. Já o controle de congestionamento emprega algoritmos como *Slow Start*, *Congestion Avoidance* e *Fast Retransmit/Fast Recovery*, que ajustam dinamicamente a janela de congestionamento. Adicionalmente, o TCP utiliza um *checksum* de 16 bits para detecção de erros no cabeçalho e no *payload* de cada segmento.

Listing 1. Pseudocódigo do Transmissor TCP

```
//      TRANSMISSOR (TCP)

1. Conectar ao servidor via TCP

2. Enviar quadro
   META(seq=0, nome_arquivo, tamanho)

3. Aguardar ACK(seq=0)
   do stream TCP

4. seq <- 1

5. Para cada bloco de 32 KB do arquivo:

       Enviar quadro
       DATA(seq, bloco)

       Aguardar ACK(seq)
       do stream TCP

       seq <- seq + 1

6. Enviar quadro FIN(seq)

7. Aguardar ACK(seq)
   do stream TCP

8. Fechar conexao
```

Listing 2. Pseudocódigo do Receptor TCP

```
//      RECEPTOR (TCP)

1. Aceitar conexao TCP

2. Para cada quadro recebido
   do stream TCP:

       Se META:

           Extrair nome/tamanho
           do JSON

           Abrir arquivo
           para escrita

           Enviar ACK(seq)

       Se DATA:

           Se seq duplicada ->
               Enviar ACK(seq)
               e descartar

           Se seq esperada ->
               Escrever payload
               no arquivo

           Enviar ACK(seq)

           seq_esperada <-
           seq_esperada + 1

       Se FIN:

           Enviar ACK(seq)

           Fechar arquivo

           Encerrar conexao
```

2.2. Protocolo UDP (User Datagram Protocol)

UDP é o segundo protocolo da camada de transporte da pilha TCP/IP e adota uma abordagem radicalmente diferente do TCP. Trata-se de um protocolo não orientado à conexão e do tipo *best-effort* (melhor esforço): não há estabelecimento de conexão prévio, não há garantia de entrega, não há controle de fluxo e não há retransmissão automática de pacotes perdidos. Seu cabeçalho é extremamente enxuto (apenas 8 *bytes*) contendo porta de origem, porta de destino, comprimento do datagrama e um *checksum* opcional. Essa simplicidade reduz o *overhead* de processamento e a latência de transmissão, tornando o UDP adequado para aplicações sensíveis a atraso.

A ausência de confiabilidade nativa do UDP implica que qualquer garantia de entrega, ordenação ou integridade deve ser implementada pela própria aplicação. Essa característica torna o UDP uma base flexível para a construção de protocolos sob medida, nos quais o desenvolvedor pode optar por implementar apenas os mecanismos de confiabilidade necessários para cada cenário, sem o *overhead* completo do TCP. É precisamente essa flexibilidade que motiva a criação de camadas de confiabilidade sobre UDP, como o R-UDP descrito na subseção seguinte.

2.2.1. R-UDP (UDP Confiável)

O R-UDP consiste em uma camada de confiabilidade implementada na própria aplicação sobre o protocolo UDP, adicionando mecanismos de entrega confiável e ordenada à comunicação originalmente não confiável do UDP. Neste trabalho, o R-UDP emprega o algoritmo *Stop-and-Wait*, o mais simples dos mecanismos de controle de fluxo, no qual o transmissor envia um pacote por vez e aguarda um ACK do receptor antes de enviar o próximo. Cada pacote possui um número de sequência de 32 bits, permitindo que o receptor detecte duplicatas. Caso um ACK se perca e o transmissor reenvie um pacote já recebido, o receptor o identifica pelo número de sequência, descarta o dado duplicado e reenvia o ACK correspondente.

O ciclo de transmissão do R-UDP ocorre nas seguintes etapas:

1. O transmissor constrói um quadro contendo a linha de autenticação *X-Custom-Auth* e um cabeçalho binário de 11 *bytes* com número de sequência, tipo da mensagem (META, DATA, ACK ou FIN), tamanho do *payload* e CRC32;
2. O datagrama é enviado por meio da chamada *sendto()*;
3. O transmissor aguarda o ACK utilizando *select()* com *timeout* fixo de 2 segundos;
4. Caso o ACK não seja recebido dentro do prazo, o pacote é retransmitido, podendo ocorrer até 1.000 tentativas antes da falha da transmissão;
5. O receptor valida a autenticidade do campo *X-Custom-Auth* e verifica a integridade do *payload* utilizando CRC32;
6. Se o CRC calculado divergir do valor recebido, o pacote é descartado, forçando a retransmissão por *timeout*.

Listing 3. Pseudocódigo do Transmissor R-UDP

```
//      TRANSMISSOR (R-UDP)

1. Criar socket UDP nao-bloqueante

2. Enviar META(seq=0) e executar
   send_sw(0, META, meta):
   Para tentativa <- 1 ate 1000:
   sendto(pacote)
   deadline <- agora + 2.0 segundos
   Enquanto agora < deadline:
       select([socket], timeout =
       deadline - agora)
       Se dados disponiveis:
           recvfrom() -> obter ACK
           Se ACK valido e seq =
           seq_esperada -> Retornar
           // Timeout: tentar novamente
           Se 1000 tentativas falharem ->
           Erro "Timeout"

3. seq <- 1

4. Para cada bloco de aproximadamente
   1000 bytes do arquivo:
       send_sw(seq, DATA, bloco) // mesmo
       mecanismo acima
       seq <- seq + 1

5. send_sw(seq, FIN, vazio)

6. Fechar socket
```

Listing 4. Pseudocódigo do Receptor R-UDP

```
//      RECEPTOR (R-UDP)

1. Criar socket UDP e bind na porta

2. Para sempre:
   recvfrom() -> obter datagrama

   Se CRC32 invalido ou X-Custom-Auth
   invalido -> Descartar

   Se META:
       Extrair nome/tamanho do JSON
       Abrir arquivo para escrita
       Enviar ACK(seq) via sendto()

   Se DATA:
       Se seq duplicada -> Enviar
       ACK(seq) e descartar dado

       Se seq esperada ->
           Escrever payload
           Enviar ACK(seq)
           seq_esperada <-
           seq_esperada + 1

       Se seq inesperada ->
           Descartar
           (timeout forcara
           retransmissao)

   Se FIN:
       Enviar ACK(seq)
       Fechar arquivo
       Retornar (bytes escritos,
       caminho)
```

3. Metodologia

Esta seção descreve a metodologia adotada para a coleta e análise dos dados experimentais. A seção 3.1 detalha o ambiente de execução, incluindo a infraestrutura Docker, a configuração de rede e os parâmetros de hardware e software utilizados. A seção 3.2 apresenta os cenários de rede simulados e o procedimento de testes. Por fim, a seção 3.3 define as métricas coletadas e descreve os métodos empregados para seu tratamento estatístico e validação cruzada com capturas de pacotes.

3.1. Ambiente de Execução

Os experimentos foram conduzidos em ambiente containerizado utilizando Docker Compose v2, com dois *containers* (servidor e cliente) interconectados por uma rede *bridge* padrão. Ambos os *containers* compartilham um volume montado que persiste os resultados no hospedeiro. A Tabela 1 resume a configuração de cada *container*.

Tabela 1. Configuração dos Containers

Componente	Função	Imagem	Porta	Privilégios
Servidor	Receptor do arquivo	Ubuntu 22.04 + Python 3.10	9000 (TCP e UDP)	–
Cliente	Transmissor do arquivo	Ubuntu 22.04 + Python 3.10	–	NET_ADMIN (para tc)

O servidor escuta na porta 9000 nos protocolos TCP e UDP, exposta para ambos os *containers*. O cliente possui a *capability* NET_ADMIN, necessária para aplicar as regras de simulação de rede via *tc qdisc* em sua interface de rede. Ambos os *containers* compartilham o volume *./results:/data*, que armazena os arquivos recebidos, as métricas CSV e as capturas de pacotes geradas durante os testes.

A Tabela 2 detalha as ferramentas e bibliotecas utilizadas no ambiente experimental.

Tabela 2. Ferramentas e Bibliotecas

Ferramenta	Versão	Função
Python	3.10	Linguagem de implementação
Docker Compose	v2	Orquestração dos containers
iproute2	–	Configuração de fila com <i>tc qdisc netem</i>
tcpdump	–	Captura de pacotes no servidor
tshark	–	Extração de métricas dos arquivos .pcap
pandas	≥ 2.0.0	Análise estatística dos resultados
matplotlib	≥ 3.7.0	Geração dos gráficos comparativos
python-dotenv	≥ 1.0.0	Carregamento das variáveis de ambiente

A Tabela 3 apresenta a configuração da máquina utilizada para a execução dos testes, fornecendo dados essenciais para a reprodutibilidade dos resultados. Essas informações permitem contextualizar os resultados obtidos, evidenciando as condições sob as quais os experimentos foram realizados.

Tabela 3. Especificações Técnicas da Máquina Utilizada nos Testes

Especificação	Descrição
Processador	Intel Core i5-12450H 12º Gen (2.00 GHz) (8 núcleos, 12 threads, 12 MB cache)
Memória RAM	16,0 GB @ 3200 MHz (utilizável: 15,7 GB)
SO	Windows 11 Home Single Language (Executado no Ubuntu 24.04.3 LTS via WSL2)

3.2. Cenários de Teste

Foram definidos três cenários de rede, simulados por meio da ferramenta *tc qdisc netem* no *container* cliente. Cada cenário combina um valor de latência (atraso de propagação) e uma taxa de perda de pacotes, conforme descrito na Tabela 4.

Tabela 4. Cenários de Rede Utilizados nos Experimentos

Cenário	Latência (ms)	Perda de Pacotes (%)
A	10	0
B	50	5
C	100	10

Para cada combinação de modo de transporte (TCP e R-UDP) e cenário (A, B e C), foram executadas 10 rodadas de transferência de um arquivo de 1 MB (Megabyte), totalizando 60 execuções.

3.3. Métricas Coletadas

Duas métricas principais foram extraídas de cada rodada. Cada uma delas é registrada simultaneamente por duas fontes independentes: a aplicação cliente, ao final da transferência, e a captura de pacotes (TCPDump) executada no servidor, permitindo a validação cruzada dos resultados. Observe a Tabela 5.

Tabela 5. Métricas Utilizadas na Avaliação

Métrica	Dimensão	Descrição Básica	Fórmula
Tempo de Transferência	s	Intervalo de tempo decorrido entre o início e o fim da transmissão	$\Delta t = t_{\text{fim}} - t_{\text{início}}$
Taxa de Transferência (Vazão/Throughput)	Mbps	Volume de dados transmitidos por unidade de tempo	$T = \frac{\text{dados enviados}}{\Delta t}$

4. Resultados

Nesta seção, são apresentados e analisados os dados obtidos nos testes de desempenho dos protocolos de transporte TCP e R-UDP aplicados à transmissão de arquivos sobre uma rede simulada com diferentes condições de qualidade de enlace. Além disso, os resultados são discutidos à luz dos fundamentos teóricos de cada abordagem, buscando relacionar o comportamento observado nos experimentos com as características esperadas em termos de tempo de transferência e vazão.

4.1. Tempo Médio de Transferência

O Gráfico 1 mostra que o TCP apresentou desempenho significativamente superior ao R-UDP em todos os cenários. No cenário A, a transferência levou em média 0,48 segundos, enquanto o R-UDP exigiu 15,45 segundos, uma diferença de aproximadamente 32 vezes. Esse resultado é esperado: o R-UDP opera no modo *stop-and-wait*, transmitindo um pacote por vez e aguardando a confirmação antes de enviar o próximo, o que introduz uma latência acumulada proporcional ao produto entre o número de pacotes e o RTT do enlace. O TCP, por sua vez, utiliza janela deslizante e ACKs cumulativos, permitindo múltiplos pacotes em trânsito simultaneamente.

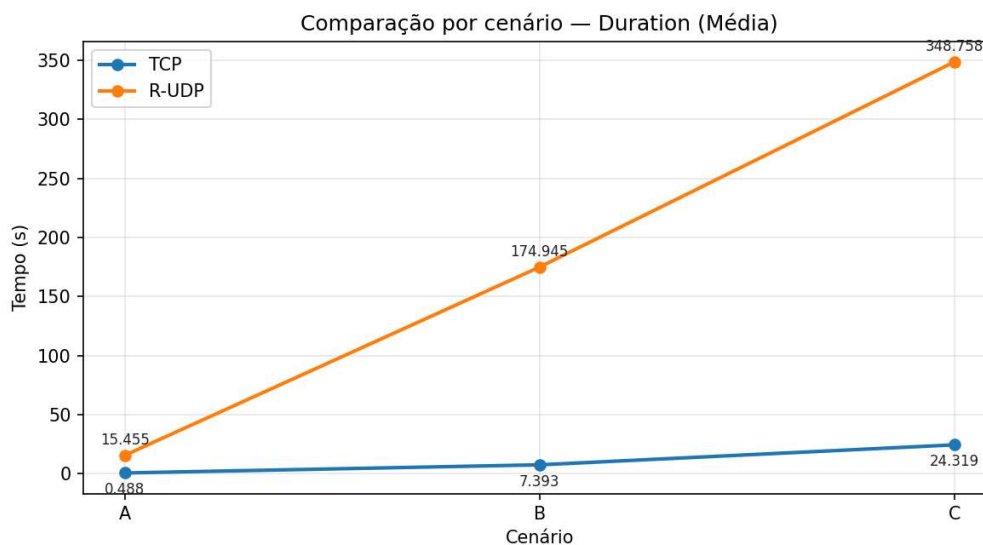


Figura 1. Comparação entre TCP e R-UDP: Tempo Médio de Transferência por Cenário

À medida que as condições de rede se deterioram (cenários B e C), a diferença entre os protocolos se reduz brevemente. No cenário C, o TCP levou 24,31 segundos contra 348,75 segundos do R-UDP, uma proporção de aproximadamente 14 vezes. A redução da vantagem relativa ocorre porque o TCP também sofre com o aumento da latência e da perda de pacotes. Sua janela de congestionamento se reduz diante de perdas, diminuindo a vazão, ainda assim, o TCP mantém vantagem expressiva em todos os cenários testados.

4.2. Vazão (*Throughput*)

As subsubseções seguintes são destinadas à análise das medidas estatísticas voltadas à vazão nos testes realizados.

4.2.1. Média

Os resultados exibidos no Gráfico 2 apresentam uma certa simetria aos de tempo de transferência, como seria esperado, pois vazão e duração são grandezas inversamente proporcionais para um mesmo volume de dados. Nos cenários com perda (B e C), a vazão de ambos os protocolos cai drasticamente, mas a vantagem relativa do TCP se reduz de 32 vezes para 14 vezes.

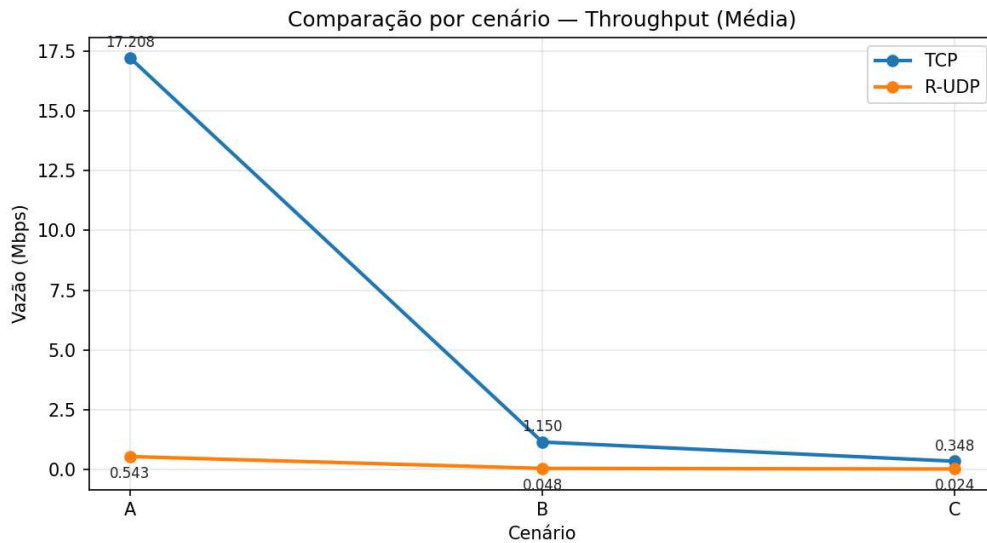


Figura 2. Comparação entre TCP e R-UDP: Vazão Média por Cenário

Isso ocorre porque o TCP reduz sua janela de congestionamento a cada perda detectada, aproximando seu comportamento do *stop-and-wait* conforme a taxa de perda aumenta. O R-UDP, por já operar nesse regime, sofre um acréscimo proporcional ao número de retransmissões, mas sem a degradação adicional da redução de janela. Ainda assim, o TCP mantém vazão superior em todos os cenários testados.

4.2.2. Vazão Mínima

A diferença entre a vazão mínima (Gráfico 3) e a média (Gráfico 2) indica o quanto o protocolo pode ser penalizado na pior rodada. No TCP, a maior discrepância ocorre no cenário A, onde a mínima (16,28 Mbps) fica 5,3% abaixo da média (17,20 Mbps), refletindo a variabilidade natural do crescimento da janela de congestionamento. Nos cenários B e C, as mínimas ficaram aproximadamente 16% e 17% abaixo da média, respectivamente, indicando que, em algumas rodadas, o TCP foi mais severamente impactado pelas perdas.

No R-UDP, a vazão mínima é muito próxima da média em todos os cenários: a diferença não ultrapassa 8% (cenário B: 0,042 Mbps vs. 0,048 Mbps). Isso reforça a observação da seção anterior: por ser um protocolo *stop-and-wait* cujo desempenho é determinado majoritariamente pelo RTT e pela taxa de perda, o R-UDP apresenta pouca variação entre rodadas, inclusive no pior caso.

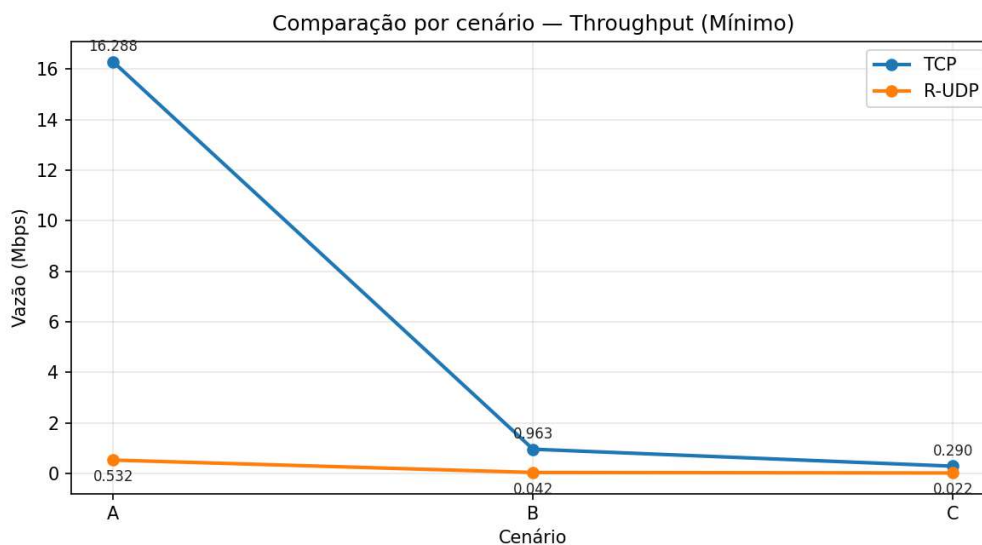


Figura 3. Comparação entre TCP e R-UDP: Vazão Mínima por Cenário

4.2.3. Vazão Máxima

O Gráfico 4 mostra que, no TCP, a vazão máxima supera a média em 8%, 26% e 13% nos cenários A, B e C, respectivamente. O destaque fica para o cenário B, em que a melhor (1,448 Mbps) foi significativamente superior à média (1,150 Mbps). Isso sugere que, em algumas execuções, o TCP conseguiu evitar perdas nos momentos críticos de crescimento da janela, alcançando uma vazão substancialmente maior que a típica para aquele cenário.

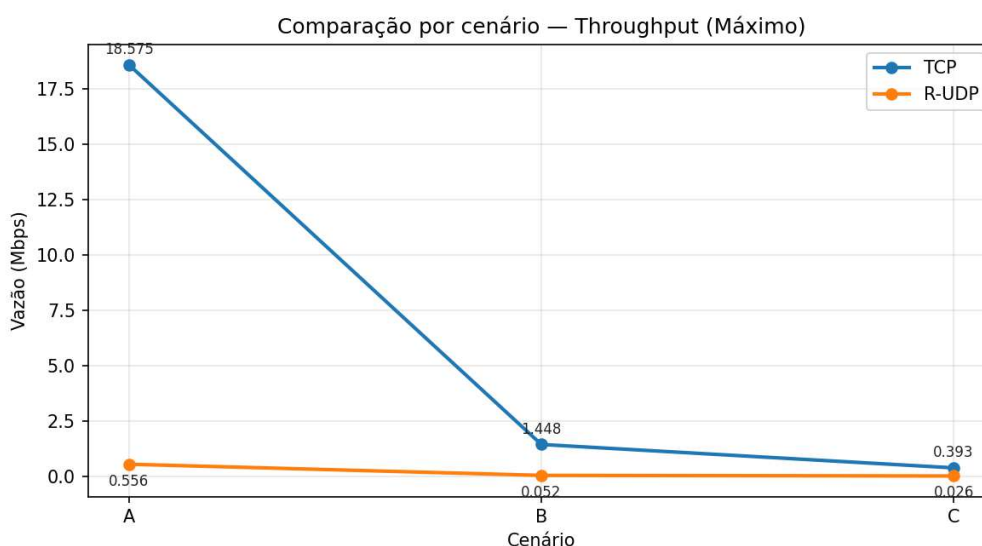


Figura 4. Comparação entre TCP e R-UDP: Vazão Máxima por Cenário

No R-UDP, novamente a máxima é muito próxima da média, com diferenças de apenas 2,5% no cenário A, 8,2% no B e 7,5% no C. Esse comportamento reforça o padrão observado: a vazão do protocolo *stop-and-wait* é estável e previsível, variando pouco

entre rodadas, mesmo no melhor caso.

4.2.4. Desvio Padrão

O Gráfico 5 revela diferenças importantes na consistência de cada protocolo. O R-UDP apresenta desvio padrão extremamente baixo em todos os cenários: 0,007 Mbps no A, 0,003 Mbps no B e 0,001 Mbps no C. Esse comportamento é esperado para o modo *stop-and-wait*: a vazão é determinada essencialmente pelo RTT e pela taxa de perda, fatores controlados deterministicamente pelo ambiente. Com pouca margem para variação entre rodadas, os valores se mantêm praticamente constantes.

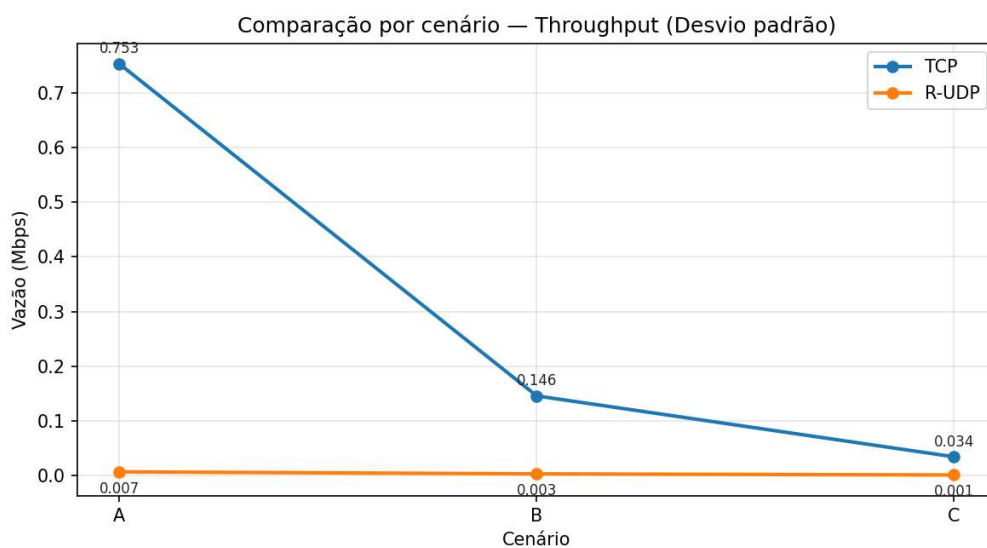


Figura 5. Comparação entre TCP e R-UDP: Desvio Padrão da Vazão por Cenário

O TCP, por outro lado, apresenta maior dispersão, especialmente no cenário A (0,753 Mbps), onde a janela de congestionamento pode crescer livremente na ausência de perdas. Pequenas variações no momento exato das chegadas dos ACKs entre as rodadas produzem diferenças no crescimento da janela, resultando em vazões menos previsíveis. Nos cenários B e C, o desvio padrão do TCP diminui (0,146 e 0,034 Mbps, respectivamente) porque a janela de congestionamento é mantida em patamares mais baixos devido às perdas, reduzindo o espaço para variação.

5. Discussão

Esta seção destina-se a responder os seguintes questionamentos:

1. Como a curva de *throughput* do TCP se comportou em comparação ao seu R-UDP sob 10% de perda?

O Gráfico 6 mostra a curva de *throughput* do TCP ao longo do tempo. Observa-se um comportamento marcado por oscilações abruptas, com picos de até aproximadamente 0,44 Mbps e quedas para valores próximos a 0,30 Mbps. Essa variação reflete o funcionamento do algoritmo de controle de congestionamento do TCP

implementado pelo *kernel* do sistema operacional. A cada perda de segmento detectada, seja por *duplicate ACK* ou por *timeout* do estimador RTO, a janela de congestionamento é reduzida, causando uma queda instantânea na vazão. Em seguida, o TCP tenta recuperar a janela por meio dos mecanismos de *slow start* ou *congestion avoidance*, gerando novos picos até a próxima perda. Esse ciclo de crescimento e colapso da janela se repete ao longo de toda a transferência, que durou em média aproximadamente 24 segundos.

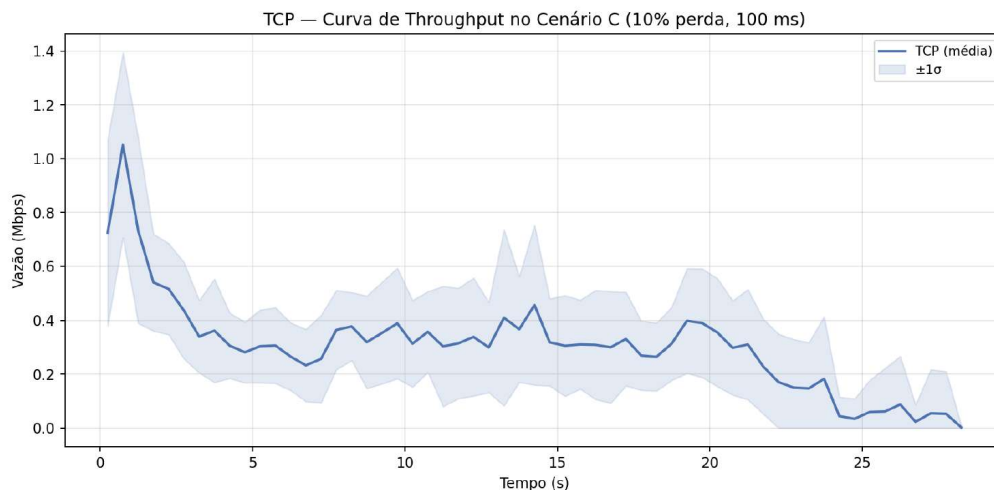


Figura 6. Curva de *Throughput* do TCP Durante a Transmissão com 10% de Perda

O Gráfico 7 apresenta a curva do R-UDP, que contrasta fortemente com a do TCP. A vazão permanece praticamente constante, em torno de 0,028 Mbps, ao longo de toda a transferência, que durou em média 348 segundos. Esse comportamento é característico do modo *stop-and-wait* implementado pelo protocolo: cada pacote de aproximadamente 1.000 *bytes* é enviado, o transmissor aguarda o ACK e só então envia o próximo.

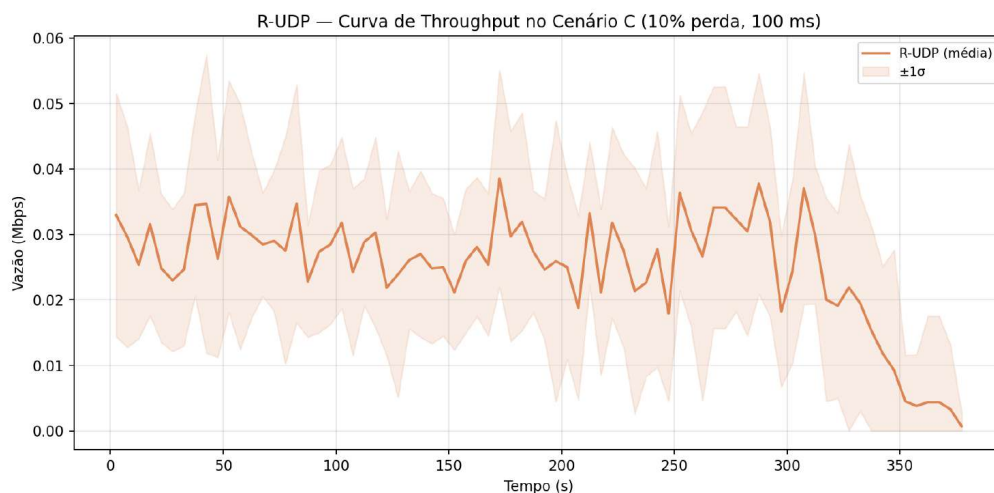


Figura 7. Curva de *Throughput* do UDP Durante a Transmissão com 10% de Perda

Não há janela deslizante nem controle de congestionamento, de modo que a vazão

instantânea é determinada unicamente pelo tamanho do pacote, pelo RTT e pela probabilidade de perda. Com 10% de perda, aproximadamente 10% dos 1.051 pacotes foram perdidos, cada um exigindo retransmissão após o *timeout* de 2,0 segundos, fator que alonga significativamente a duração total, mas mantém a vazão estável.

O Gráfico 8 sobrepõe as curvas dos dois protocolos em tempo normalizado, permitindo a comparação direta do formato de cada curva independentemente da diferença de escala temporal. O TCP exhibe um perfil ondulatório, alternando entre picos e vales ao longo de toda a duração da transferência, enquanto o R-UDP mantém uma linha essencialmente reta. A faixa de ± 1 desvio padrão, representada pela área sombreada ao redor de cada curva, revela ainda que o R-UDP é mais consistente entre rodadas (faixa estreita), enquanto o TCP apresenta maior variabilidade, consequência direta da natureza probabilística das perdas e da resposta adaptativa do controle de congestionamento.

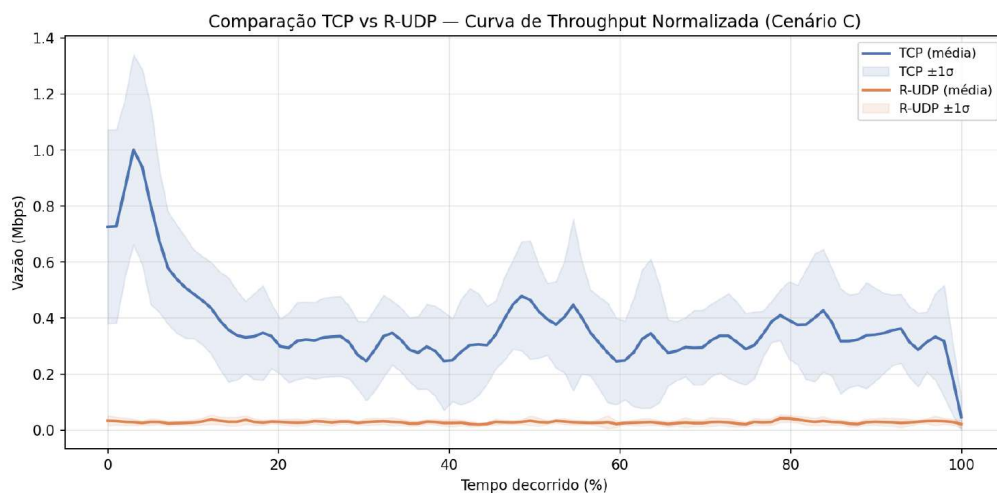


Figura 8. Comparação da Curva de *Throughput* Normalizada entre TCP e R-UDP Durante a Transmissão com 10% de Perda

Em termos quantitativos, o TCP registrou vazão média de 0,348 Mbps, contra 0,024 Mbps do R-UDP, uma vantagem de aproximadamente 14 vezes. Embora ambos os protocolos sofram degradação superior a 95% em relação ao Cenário A (sem perdas), o mecanismo de janela deslizante e controle de congestionamento do TCP permite que múltiplos segmentos trafeguem simultaneamente, mantendo vazão superior mesmo sob condições adversas. O R-UDP, por operar em regime estritamente serial, um pacote por vez, tem sua vazão limitada pela razão entre o tamanho do pacote e o RTT, agravada pelo *timeout* fixo de 2,0 segundos que penaliza cada pacote perdido.

Esses resultados evidenciam que, embora o TCP seja mais suscetível a oscilações de vazão sob perda, sua capacidade de adaptação dinâmica lhe confere desempenho significativamente superior ao do R-UDP em cenários de rede adversos.

2. Ao analisar o Wireshark/TCPDump, qual foi o *overhead* (em bytes) introduzido pelos cabeçalhos do seu protocolo R-UDP?

Cada datagrama R-UDP é composto por três partes na camada de aplicação: uma linha de autenticação X-Custom-Auth, seguida do *hash* SHA-256 (81 bytes), um

cabeçalho binário com número de sequência, tipo, tamanho do *payload* e CRC32 (11 *bytes*), e o *payload* propriamente dito. O *overhead* fixo introduzido pelos cabeçalhos do protocolo é de 92 *bytes* por datagrama.

Para pacotes DATA com carga útil de 1.000 *bytes*, o *overhead* representa 8,4% do total (1.092 *bytes* de UDP *payload*). Para pacotes de controle como ACK e FIN, que possuem *payload* vazio, o *overhead* é de 100%, o datagrama inteiro é composto apenas pelos cabeçalhos, totalizando 92 *bytes*. Para fins de comprovação, observe a figura 9 no **Anexo I** (página 17).

3. **Houve discrepância entre o tempo medido pelo Python e o registrado no Wireshark/TCPDump? Justifique.**

As Tabelas 6 e 7 apresentadas no **Anexo II** (página 18) mostram que o tempo medido pela aplicação é sistematicamente maior que o registrado pelo pcap em todas as rodadas, com diferenças que variam de +2,2% a +12,6%. Essa discrepância é esperada e pode ser atribuída a dois fatores:

- (a) Escopo da medição: a aplicação mede o tempo total de execução, incluindo a criação do *socket*, a serialização dos metadados e o fechamento das conexões, operações que ocorrem antes do primeiro pacote ser enviado e depois do último ser recebido e, portanto, não são capturadas pelo pcap.
- (b) Atraso entre *sendto()* e o *timestamp* da captura: quando a aplicação chama *sendto()*, os dados são copiados para o *buffer* do *kernel*. O pcap registra o *timestamp* apenas quando o pacote é efetivamente processado pela interface de rede. Esse atraso, da ordem de microssegundos a milissegundos por pacote, acumula-se linearmente com o número de pacotes transmitidos.

A magnitude da diferença é maior nos cenários com menor duração total (TCP, Cenário A: +12,6%) porque o tempo de *setup/teardown* do *socket* representa uma fração proporcionalmente maior do tempo total. Nos cenários mais longos (R-UDP, Cenário C: +2,2%), essa fração se dilui. Conclui-se que a discrepância existe, é consistente, de magnitude pequena a moderada (entre 2% e 13%), e é explicável por fatores inerentes à instrumentação, não por erros de implementação ou medição.

6. Conclusão

Este trabalho implementou e comparou dois protocolos de transferência de arquivos, TCP e R-UDP (UDP com camada de confiabilidade), sob três cenários de rede simulados com tc netem: A (10ms, 0% de perda), B (50 ms, 5%) e C (100ms, 10%). Foram realizadas 10 rodadas de transferência de 1 MB para cada combinação, totalizando 60 execuções.

O TCP mostrou-se significativamente superior em todos os cenários. No Cenário A, atingiu vazão média de 17,20 Mbps (0,48s), contra 0,54 Mbps (15,45s) do R-UDP, aproximadamente 32 vezes mais rápido. A diferença acentuou-se com a perda de pacotes: no Cenário C, o TCP completou em 24,32s (0,34 Mbps), enquanto o R-UDP levou 348,76s (0,024Mbps). Esse comportamento deve-se ao mecanismo *Stop-and-Wait* do R-UDP, que mantém apenas um pacote em trânsito e utiliza *timeout* fixo de 2 segundos, tornando-o vulnerável à latência e à perda. O TCP, por sua vez, emprega janela deslizante, *timeout* adaptativo e retransmissão rápida, que mitigam esses efeitos.

O *overhead* do R-UDP foi de 92 *bytes* por datagrama (81 de autenticação + 11 de cabeçalho binário), representando 8,4% de cada pacote DATA e resultando em eficiência de 91,6% na transferência. A validação cruzada com *tshark* confirmou os valores teóricos e revelou uma discrepância sistemática de +2% a +13% entre o tempo medido pela aplicação e o registrado pelo pcap, atribuída a fatores de instrumentação como *setup* do *socket* e atraso entre *sendto()* e o *timestamp* da captura.

Os resultados evidenciam as limitações do R-UDP e demonstram, na prática, por que protocolos consolidados como TCP utilizam mecanismos mais sofisticados de controle de fluxo e congestionamento para operar eficientemente em condições adversas de rede.

Anexo I

Capturas Complementares: Comparação Entre Pacotes do Protocolo R-UDP

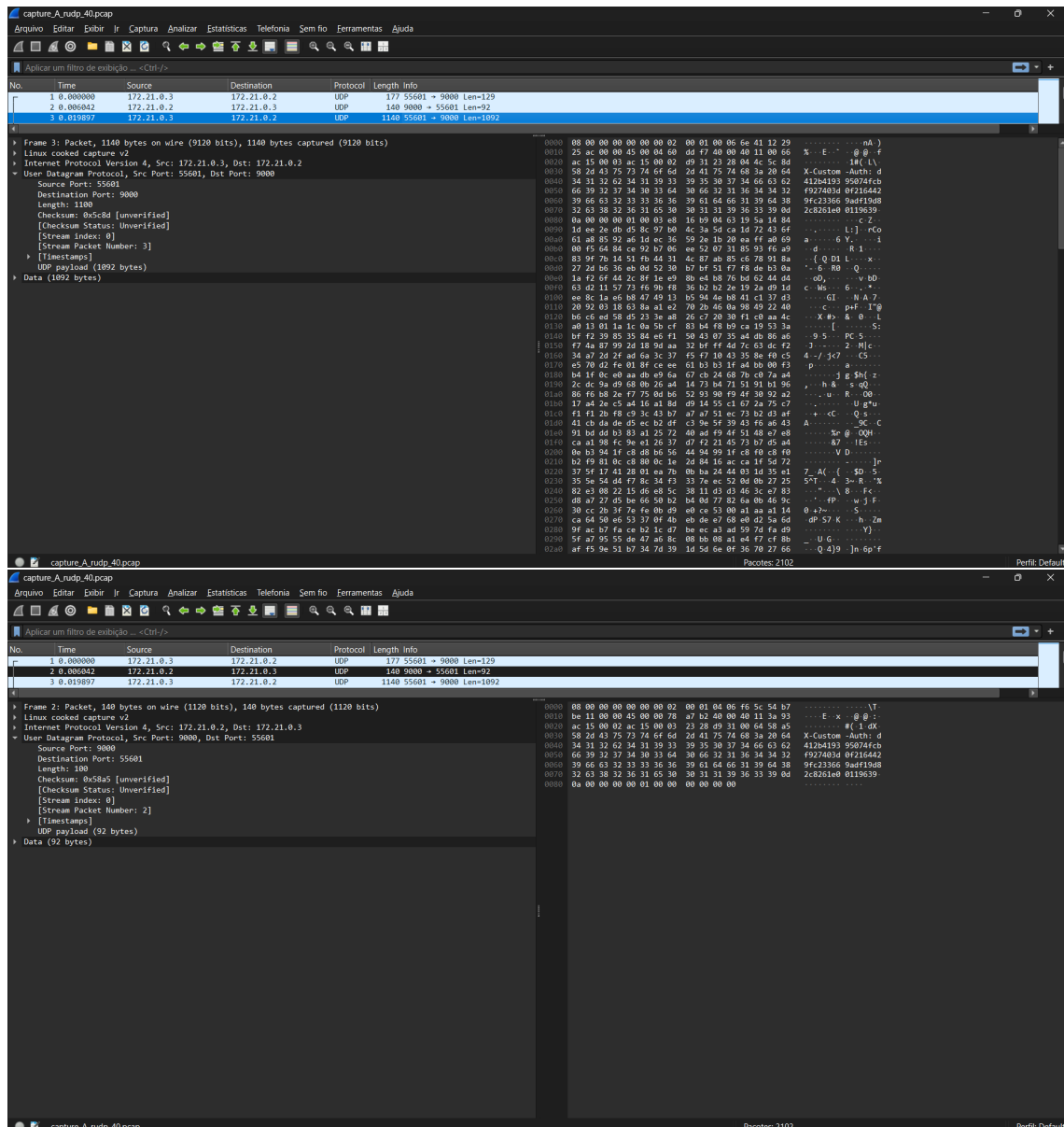


Figura 9. Comparação entre pacotes DATA e ACK do protocolo R-UDP no Wireshark.

Anexo II

Tabelas Complementares: Comparação App vs. Wireshark/TCPDump

Tabela 6. Comparação do Tempo de Transferência por Execução: Aplicação vs. Wireshark/TCPDump

Run	C	Modo	App (s)	Wireshark/TCPDump (s)	Dif. (%)
1	A	TCP	0,5150	0,4510	+14,18
2	A	TCP	0,4613	0,4095	+12,63
3	A	TCP	0,4516	0,3980	+13,47
4	A	TCP	0,4945	0,4387	+12,72
5	A	TCP	0,4756	0,4244	+12,08
6	A	TCP	0,5012	0,4471	+12,09
7	A	TCP	0,4765	0,4267	+11,69
8	A	TCP	0,5039	0,4465	+12,85
9	A	TCP	0,4985	0,4448	+12,06
10	A	TCP	0,5049	0,4496	+12,29
11	B	TCP	7,1336	6,4389	+10,79
12	B	TCP	8,4178	8,2611	+1,90
13	B	TCP	7,5481	7,1470	+5,61
14	B	TCP	6,9191	6,9206	-0,02
15	B	TCP	5,7927	5,4126	+7,02
16	B	TCP	8,0325	6,5582	+22,48
17	B	TCP	6,4355	6,1777	+4,17
18	B	TCP	7,6635	7,5037	+2,13
19	B	TCP	7,2762	6,5402	+11,25
20	B	TCP	8,7133	8,3277	+4,63
21	C	TCP	28,8880	28,2416	+2,29
22	C	TCP	21,4194	20,6784	+3,58
23	C	TCP	23,3259	22,6534	+2,97
24	C	TCP	22,9633	21,6428	+6,10
25	C	TCP	25,1417	24,0735	+4,44
26	C	TCP	22,7221	22,0249	+3,17
27	C	TCP	27,8665	27,0727	+2,93
28	C	TCP	21,3539	19,5849	+9,03
29	C	TCP	24,5585	24,2312	+1,35
30	C	TCP	24,9549	24,1044	+3,53
31	A	R-UDP	15,5986	14,8605	+4,97
32	A	R-UDP	15,5155	14,7080	+5,49
33	A	R-UDP	15,7632	15,6873	+0,48
34	A	R-UDP	15,4669	14,7142	+5,12
35	A	R-UDP	15,2399	14,4973	+5,12
36	A	R-UDP	15,4290	15,4174	+0,08
37	A	R-UDP	15,5062	14,7483	+5,14
38	A	R-UDP	15,6230	15,4585	+1,06
39	A	R-UDP	15,0781	15,0664	+0,08
40	A	R-UDP	15,3256	14,5323	+5,46
41	B	R-UDP	182,9001	178,2122	+2,63
42	B	R-UDP	185,1462	180,5431	+2,55
43	B	R-UDP	177,2509	172,5493	+2,72
44	B	R-UDP	179,2426	174,8888	+2,49
45	B	R-UDP	161,1121	157,1540	+2,52
46	B	R-UDP	201,4696	195,9049	+2,84
47	B	R-UDP	161,0082	157,0060	+2,55
48	B	R-UDP	169,0941	164,9043	+2,54
49	B	R-UDP	165,0498	160,2465	+3,00
50	B	R-UDP	167,1740	163,1537	+2,46
51	C	R-UDP	327,3437	318,3966	+2,81
52	C	R-UDP	333,2664	324,4747	+2,71
53	C	R-UDP	323,3930	314,4514	+2,84
54	C	R-UDP	367,7368	358,5217	+2,57
55	C	R-UDP	359,4423	349,7128	+2,78
56	C	R-UDP	344,7161	337,5320	+2,13
57	C	R-UDP	386,6253	379,3069	+1,93
58	C	R-UDP	340,7027	334,1160	+1,97
59	C	R-UDP	356,2776	352,4191	+1,09
60	C	R-UDP	348,0801	343,5960	+1,31

Tabela 7. Comparação do Tempo Médio de Transferência: Aplicação vs. Wireshark/TCP-Dump

Cenário	Modo	App (média)	Wireshark/TCPDump (média)	Diferença
A	TCP	0,49 s	0,43 s	+12,6%
B	TCP	7,39 s	6,93 s	+7,0%
C	TCP	24,32 s	23,43 s	+3,9%
A	R-UDP	15,45 s	14,97 s	+3,3%
B	R-UDP	174,94 s	170,26 s	+2,6%
C	R-UDP	348,76 s	341,25 s	+2,2%